Universidad Nacional de Educación a Distancia
(UNED)

Máster Universitario en Investigación en Inteligencia
Artificial

MEMORIA DE ACTIVIDAD

Modelado y Consulta de Property-Graphs con Neo4j

Asignatura:

Web Semántica y Enlazado de Datos

Autor
Fernando Martínez Marco

Fecha
3 de febrero de 2026

Curso Académico 2025/2026

Índice

1. Introducción **2**

2. Diseño del Grafo **3**

3. Código Cypher **4**

4. Configuración y análisis **6**

4.1. Reglas de integridad y velocidad 6

4.2. Sugerencia de Grupos de Estudio 7

4.3. Caminos y cálculos 7

5. Conclusión y Comparativa **9**

1. Introducción

En esta actividad exploramos el modelo de **Property-Graphs**, una alternativa a RDF muy usada. Con **Neo4j** y **Cypher** diseñamos un modelo universitario donde guardamos información tanto en los nodos como en las relaciones.

A diferencia de OWL, que es más estricto con la lógica, los Property-Graphs son más flexibles y rápidos para navegar por el grafo. Así pudimos modelar de forma sencilla las notas de los alumnos o los requisitos entre asignaturas.

2. Diseño del Grafo

Para esta práctica usamos el mismo ejemplo de la universidad pero adaptado a este modelo de propiedades. El modelo tiene:

- **Nodos**: Etiquetados como `:Persona`, `:Estudiante`, `:Profesor`, `:Course` y `:Degree`.
- **Relaciones**: `ENROLLED_IN` (con nota), `TEACHES` (con semestre), `REQUIRES` (prerrequisitos) y `HAS_DEGREE`.

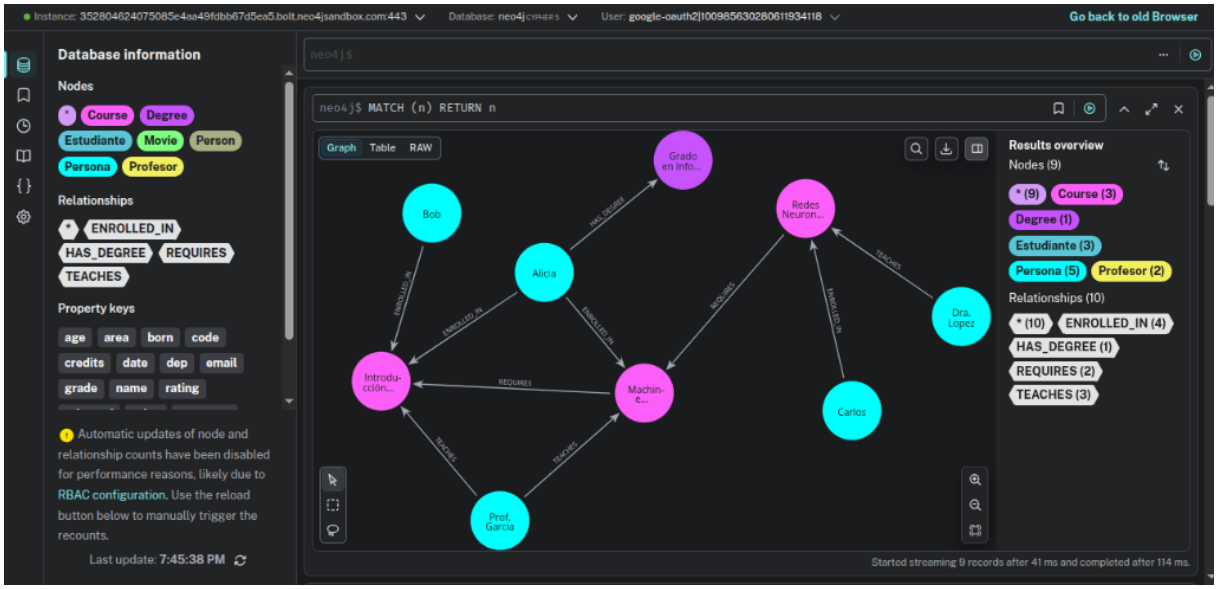


Figura 1: Captura del grafo con sus nodos y relaciones principales.

3. Código Cypher

El siguiente script incluye la creación de los datos, las reglas de integridad y las consultas que probamos.

```
// --- 1. Inicialización ---
MATCH (n) DETACH DELETE n;

// --- NUEVO: Integridad de datos ---
CREATE CONSTRAINT IF NOT EXISTS FOR (c:Course) REQUIRE c.code IS UNIQUE;
CREATE INDEX IF NOT EXISTS FOR (s:Estudiante) ON (s.name);


// --- 2. CREACIÓN DEL MODELO (Nodos y Propiedades) ---
CREATE (p1:Persona:Profesor {name: 'Prof. Garcia', dep: 'IA', seniority: 10})
CREATE (p2:Persona:Profesor {name: 'Dra. Lopez', dep: 'Matemáticas', seniority: 5})

CREATE (s1:Persona:Estudiante {name: 'Alicia', age: 22, email: 'alicia@ejemplo.org'})
CREATE (s2:Persona:Estudiante {name: 'Bob', age: 24, email: 'bob@ejemplo.org'})
CREATE (s3:Persona:Estudiante {name: 'Carlos', age: 21, email: 'carlos@ejemplo.org'})
CREATE (s4:Persona:Estudiante {name: 'David', age: 23, email: 'david@ejemplo.org'})

CREATE (c1:Course {title: 'Introducción a la IA', code: 'IA101', credits: 6})
CREATE (c2:Course {title: 'Machine Learning', code: 'ML202', credits: 6})
CREATE (c3:Course {title: 'Redes Neuronales', code: 'DL303', credits: 9})

CREATE (d1:Degree {name: 'Grado en Informática', area: 'Ingeniería'});


// --- 3. GESTIÓN DE RELACIONES (Con atributos) ---
// Relaciones Académicas
CREATE (p1)-[:TEACHES {semester: 1, year: 2024}]->(c1)
CREATE (p1)-[:TEACHES {semester: 2, year: 2024}]->(c2)
CREATE (p2)-[:TEACHES {semester: 1, year: 2024}]->(c3)

// Matriculaciones con Notas (Calificaciones en la relación)
CREATE (s1)-[:ENROLLED_IN {grade: 9.5, date: '2024-09-01'}]->(c1)
CREATE (s1)-[:ENROLLED_IN {grade: 8.0, date: '2024-09-01'}]->(c2)
CREATE (s2)-[:ENROLLED_IN {grade: 7.0, date: '2024-09-01'}]->(c1)
CREATE (s3)-[:ENROLLED_IN {grade: 6.5, date: '2024-09-01'}]->(c3)
CREATE (s4)-[:ENROLLED_IN {grade: 9.0, date: '2024-09-01'}]->(c1)
```

```

// Prerrequisitos (Navegación recursiva)
CREATE (c2)-[:REQUIRES]->(c1)
CREATE (c3)-[:REQUIRES]->(c2);


// --- 4. EXPERIMENTACIÓN Y CONSULTA (Cypher) ---

// Q1: Análisis de Desempeño (Agregación)
// ¿Cuál es la nota media de cada curso?
MATCH (s:Estudiante)-[r:ENROLLED_IN]->(c:Course)
RETURN c.title AS Curso, avg(r.grade) AS MediaNotas, count(s) AS Alumnos
ORDER BY MediaNotas DESC;

// --- NUEVO: Recomendación de Grupos de Estudio ---
MATCH (s1:Estudiante)-[r1:ENROLLED_IN]->(c:Course)<-[r2:ENROLLED_IN]-(s2:Estudiante)
WHERE s1 <> s2
  AND abs(r1.grade - r2.grade) < 1.0 // Alumnos con notas similares
RETURN s1.name, s2.name, c.title AS CursoCompartido, "Potencial grupo de estudio" AS Motivo;

// Q3: Trazabilidad de Conocimiento (Exploración de Caminos)
// ¿Qué asignaturas necesito superar para llegar a Redes Neuronales?
MATCH path = (c:Course {title: 'Redes Neuronales'})-[:REQUIRES*]->(base:Course)
RETURN nodes(path) AS CadenaPrerrequisitos;

// Q4: Descubrimiento de Influencia (Conectividad)
// ¿Quiénes son los estudiantes que han cursado asignaturas de más de un departamento?
MATCH (s:Estudiante)-[:ENROLLED_IN]->(c:Course)<-[:TEACHES]-(p:Profesor)
WITH s, count(DISTINCT p.dep) AS deps
WHERE deps > 1
RETURN s.name, deps;
```

Listing 1: Script de administración y consulta

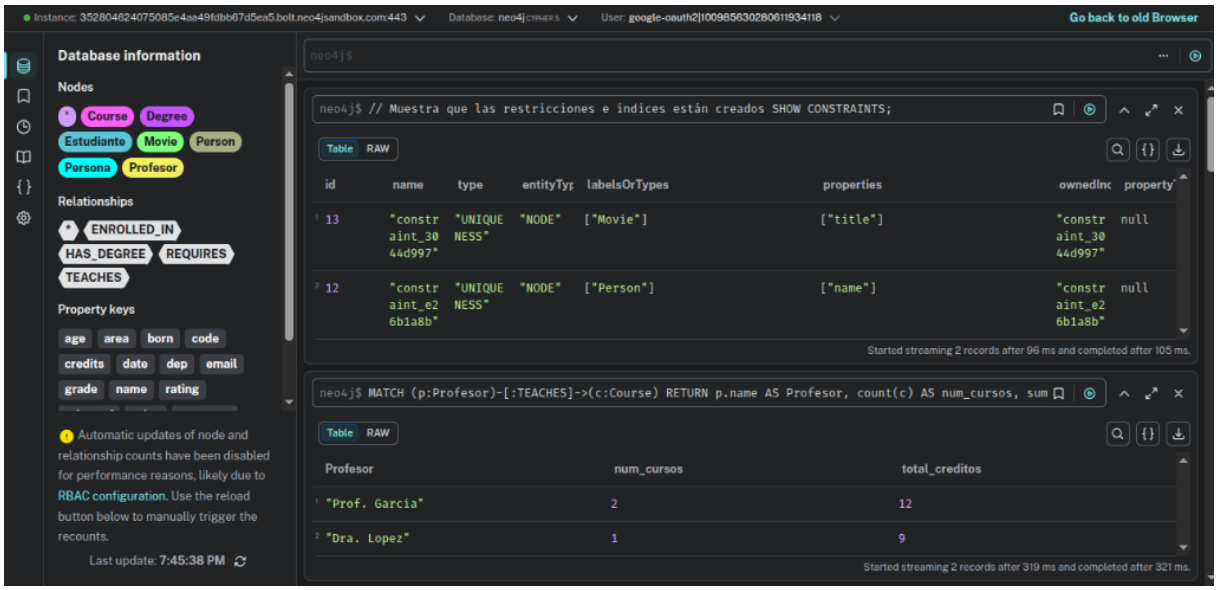
4. Configuración y análisis

Además del modelo básico, añadimos algunas reglas para que el grafo funcione mejor.

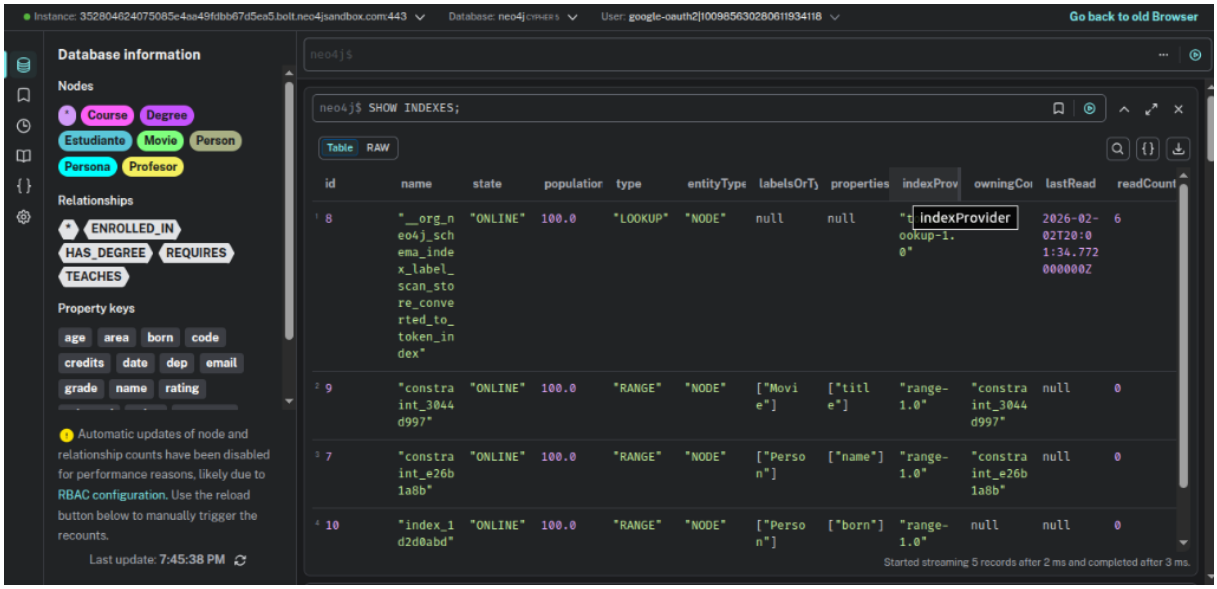
4.1. Reglas de integridad y velocidad

Para evitar errores al cargar los datos y que todo funcione más rápido, hemos configurado:

- **Restricciones de Unicidad**: Con `CREATE CONSTRAINT` nos aseguramos de que no haya dos cursos con el mismo código.
- **Uso de Índices**: Creamos un índice para el nombre del estudiante. Así las búsquedas son rápidas aunque el grafo crezca (ver Figura 2).



(a) Capturas de las restricciones creadas.



(b) Capturas de los índices funcionando.

Figura 2: Gestión de la base de datos: capturas que muestran la integridad y la optimización de las búsquedas.

4.2. Sugerencia de Grupos de Estudio

Hicimos una consulta que sugiere posibles **grupos de estudio** buscando alumnos del mismo curso con notas parecidas. Es una forma sencilla de generar recomendaciones automáticas.

4.3. Caminos y cálculos

También probamos cómo Neo4j maneja rutas y cálculos. Por ejemplo, para ver los requisitos de una asignatura (Figura 3) o sumar los créditos de un profesor (Figura 4).

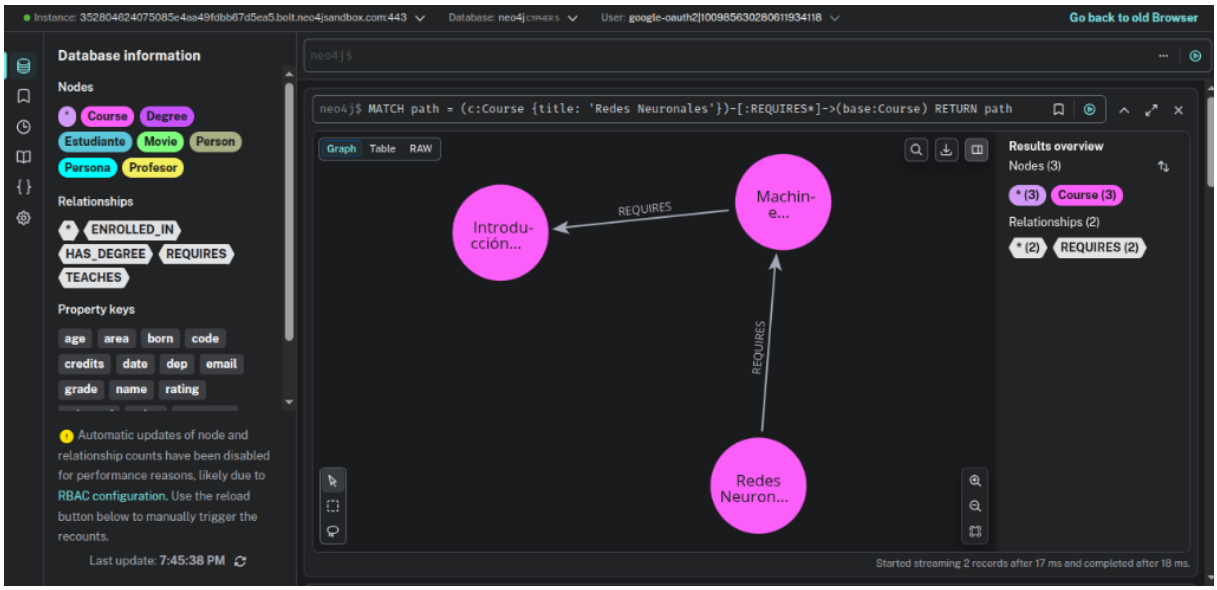


Figura 3: Trazabilidad de conocimiento recorriendo la cadena de prerrequisitos.

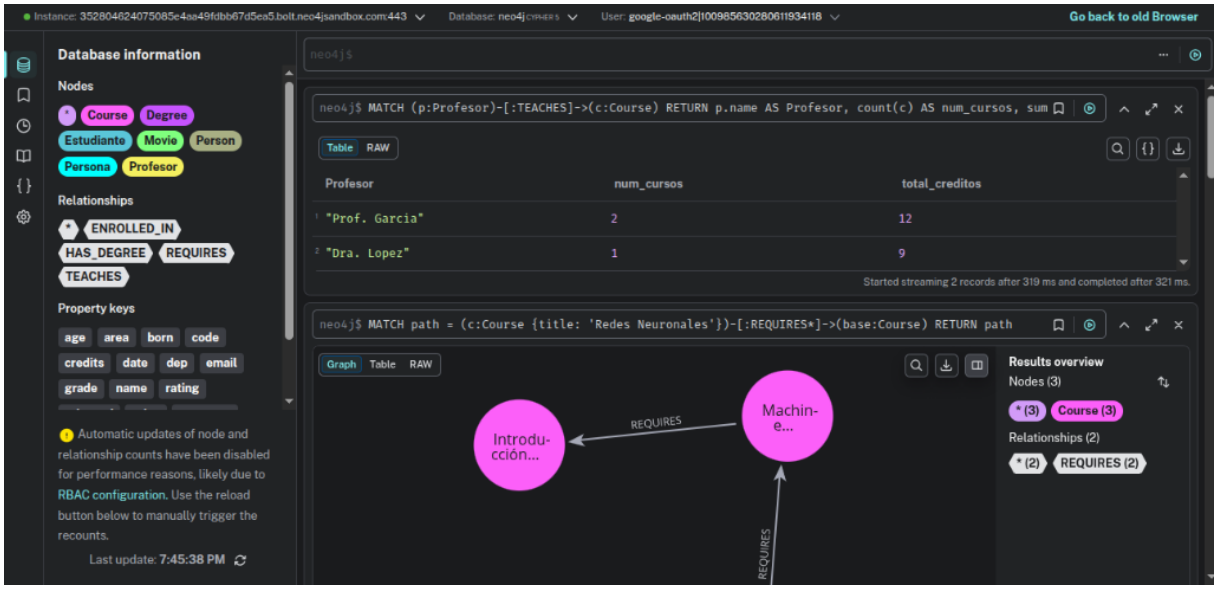


Figura 4: Resultados analíticos y tablas de agregación obtenidas mediante Cypher.

5. Conclusión y Comparativa

Tras pasar por RDF, OWL y finalmente los Property-Graphs, comparamos ambas tecnologías. Cada una sirve para una cosa distinta:

Característica	**RDF/OWL (Semántica)**	**Property-Graph (Neo4j)**
Objetivo	Interoperabilidad y Lógica	Rendimiento y Flexibilidad
Esquema	Basado en Ontologías (Rígido)	Dinámico (Sin esquema prefijado)
Propiedades	Solo en nodos (como tripletas)	En nodos y en relaciones
Razonamiento	Automático (Motor de Inferencia)	Manual (Lógica en Cypher)
Escalabilidad	Compleja en grafos muy densos	Alta para navegación de saltos

Cuadro 1: Comparativa técnica: Web Semántica vs. Bases de Datos de Grafos.

En conclusión, ambas tecnologías son útiles. OWL es mejor para compartir datos con un significado claro, mientras que Neo4j es mucho más rápido y práctico para analizar conexiones.